SALESTORM

FLASH-SALE PLATFORM BUILT FOR EXTREME CONCURRENCY

10,000 BUYERS. 100 UNITS. ZERO OVERSELL.

ZERO OVERSELL | IDEMPOTENT PAYMENTS | RESILIENT ORDER PROCESSING

SYSTEM DESIGN HACKATHON | SYSCRAFTERS 2026

TEAM NAME: DIGITAL DYNAMOS

MEMBERS: NARENDHRAN V.S
PRAVEEN K
SARAN S

REQUIREMENTS AND SIZING

10,000 TARGET req/sec sustained	**500,000** TARGET req/sec edge burst	**200,000+** TARGET inventory gate ops/sec
UNDER 50 MS TARGET fast gate reserve	**UNDER 200 MS** TARGET end-to-end latency	**99.9%** TARGET platform uptime

STRICTGUARANTEES	ZERO OVERSELL	ZERO DOUBLE CHARGES	10-MINUTE HOLDS	ZERO LOST ORDERS

HIGH-LEVEL ARCHITECTURE

CLOUDFLARE EDGE NETWORK

- CDN
- WAF
- WAITING ROOM

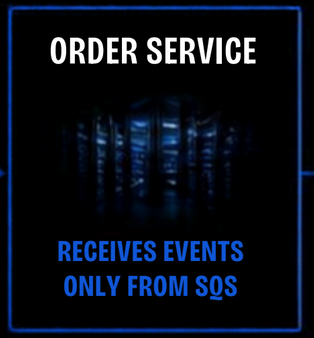
500,000 BURST
normalized at the edge

API GATEWAY

- RATE LIMITING

CORE SERVICES

- CATALOG
- INVENTORY
- CHECKOUT



500,000 BURST
normalized at the edge

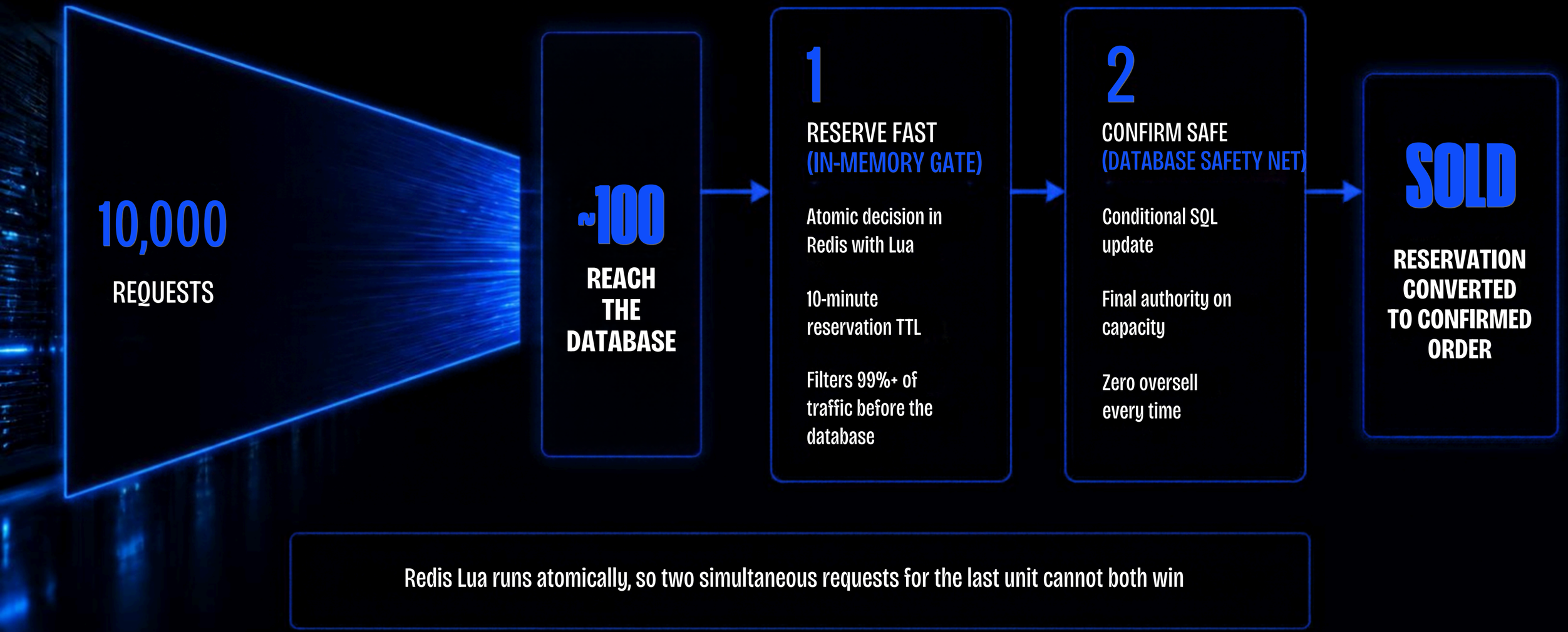
FAST GATE: REDIS
protects the core

SOURCE OF TRUTH: POSTGRESQL
ensures data integrity

RELIABLE ASYNC
outbox plus SQS for resilient delivery

THE CRITICAL PATH

TWO-PHASE GATE KEEPS THE SALE FAIR, FAST, AND SAFE UNDER EXTREME LOAD



Redis Lua runs atomically, so two simultaneous requests for the last unit cannot both win

PAYMENT RESILIENCE

RELIABLE PAYMENTS. NO DATA LOSS. ZERO DOUBLE CHARGES.

01
CHECKOUT
customer places order

→

02
PAYMENT
idempotency key prevents double charges

→

03
ACID TRANSACTION
payment and outbox event committed atomically

→

04
OUTBOX
event persisted and ready

→

05
SQS QUEUE
event held while Order Service is down

→

06
ORDER RECOVERY
Order Service recovers and processes the event

OUTAGE SCENARIO

30 SECONDS

Order Service is unavailable

SQS holds the event securely

No orders lost

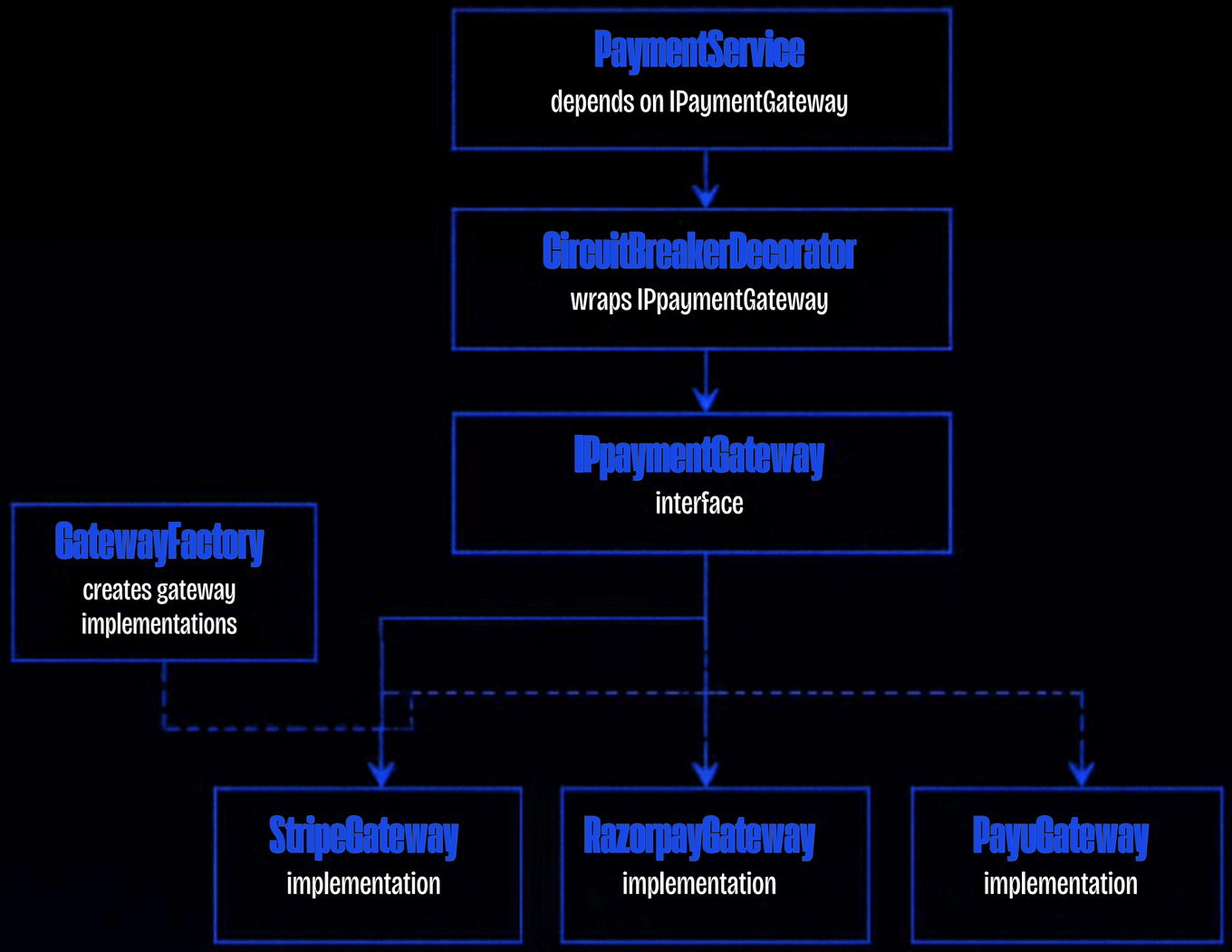
No payments duplicated

Processing resumes automatically

PAYMENT FAILS
reservation released, stock returns to the pool

TIMEOUT
query the gateway by reference, never re-charge

LOW-LEVEL DESIGN AND PATTERNS



STRATEGY
pluggable payment gateways

DECORATOR
circuit breaker for resilience

FACTORY
creates gateway implementations

OBSERVER
outbox publishes events to SQS

FACADE
CheckoutService simplifies flow

DEPENDENCY INVERSION
PaymentService depends on abstraction, not concrete

NEW PAYMENT PROVIDER
means one new class, no core changes

STRESS TEST

10,000 BUYERS AND 100 UNITS SIMULATION

90
CONFIRMED SOLD

10
RELEASED BACK TO STOCK

0
OVERSOLD

IDEMPOTENCY AND STOCK ACCURACY:

PASS

0.48S
SIMULATION COMPLETED

9,856
FAST-GATE REJECTS

PAYMENT SUCCESS 95%, FAILURE 5%, DUPLICATES 2%, ORDER SERVICE DOWN

30 SECONDS

ZERO UNITS OVERSOLD

THE SIMULATION VALIDATES DESIGN LOGIC, NOT REAL INFRASTRUCTURE SPEED.

50X BURST

500k to 10k	edge waiting room 500k to 10k req/sec at origin
50-100	horizontal service autoscaleing 4 to 50-100 tasks
300k+	Redis cluster 300k+ ops/sec target
100-200	PostgreSQL writes capped at 100-200 TPS

THE SYSTEM SCALES USERS WITHOUT SCALING DATABASE CONTENTION.

FAILURE RECOVERY

1 REDIS FAILS

Fall back to the
SQL conditional update

↓

Rebuild the Redis counter from
the database

↓

Slower but still
zero oversell

2 DATABASE FAILS

Multi-AZ failover
in under 60 seconds
(target)

↓

Writes retried

↓

Reservations keep their expiry times

3 PAYMENT GATEWAY FAILS

Circuit breaker opens after 5
failures

↓

Fail fast

↓

Queue the request

↓

Probe in half-open
state after 30 seconds

4 ORDER SERVICE FAILS

Event waits in SQS

↓

Retries

↓

Dead-letter queue and alert as
last resort

↓

Refund through reconciliation only if
the order cannot be created

DUPLICATES AND SOLD OUT

DUPLICATE BUY CLICK

1. Same idempotencykey from customer,sale and product
2. Unique constraint rejects the second reservation
3. Customer receives the original result

DUPLICATE PAYMENT

1. Same payment key
2. Gateway call replayed safely
3. One charge only

INVENTORY REACHES ZERO

1. Redis counter at 0
2. Returns **Sold Out** instantly
3. Database never touched

LAST UNIT RACE

1. Two requests arrive together
2. One atomic Lua decrement
3. One winner, one **Sold Out**

ARCHITECTURE TRADE-OFFS

	DECISION	CHOSEN OPTION	WHAT IT SACRIFICES
1	CONCURRENCY	Redis Lua gate plus SQL conditional update	Redis and database can briefly disagree, reconciled by the sweeper
2	DATABASE	PostgreSQL over NoSQL for ACID inventory	Harder horizontal write scaling, mitigated because writes stay near 100 to 200
3	PROCESSING	Sync for reserve and pay, async for order, shipping and notifications	Eventual consistency for orders
4	DELIVERY	Transactional outbox plus SQS	More moving parts, at-least-once delivery needs idempotent consumers

SECURITY AND OBSERVABILITY

SECURITY

JWT RS256 authentication and
role-based authorization

TLS 1.3 everywhere, WAF and
rate limiting at the edge and per user

Input validation at the gateway,
tokenized payments, no card data stored

Audit log on every state change,
secrets in a managed vault

OBSERVABILITY

Metrics: request rate, latency, error rate,
reservation failures, payment failures, order conversion

Structured business-event logs and distributed tracing across checkout,
payment and order

Alerts: inventory mismatch between Redis and SQL, payment failure spike,
SQS queue backlog

SOLID AND RESPONSIBLE AI USE

SOLID IN PRACTICE

1. ExpirySweeper separate from InventoryService
2. New gateway as a new class, no core edits
3. Any gateway is substitutable for IPaymentGateway
4. Small focused interfaces for repositories and gateways
5. PaymentService depends on interfaces, not SDKs

RESPONSIBLE AI USE

1. Antigravity and Claude for diagram drafts, documentation and the simulation script
2. Design decisions, concurrency reasoning and trade-offs made by the team
3. All AI output reviewed, tested and explained by every member